

UNIVERSIDAD CATÓLICA SAN PABLO
DEPARTAMENTO DE COMPUTACIÓN
CURSO: INGENIERÍA DE SOFTWARE II — SEMESTRE 2026-I

INFORME DE LABORATORIO 06: PRUEBAS DE RENDIMIENTO

Automatización y Cobertura de Carga Masiva mediante Apache JMeter

Docente: DSc. Edgar Sarmiento Calisaya

Estudiante: [Ingresar Nombre y Apellidos]

Fecha de Entrega: 11 de junio de 2026

1 Objetivos del Laboratorio

- Automatizar las pruebas de rendimiento dinámicas sobre una arquitectura de servicios web utilizando Apache JMeter.
- Maximizar la cobertura de pruebas de caja negra simulando flujos concurrentes estructurados de consulta (GET) y mutación de estado (POST).
- Diagnosticar la estabilidad del servidor web Java Spring Boot bajo estrés concurrente mediante aserciones automáticas de tiempo límite.
- Recopilar, procesar y evaluar las métricas clave de rendimiento mediante la generación automática de tableros analíticos interactivos (Dashboard HTML).

2 Configuración del Entorno de Pruebas

El entorno experimental se desplegó localmente para garantizar el aislamiento de red y la reproducibilidad de las mediciones:

- **Aplicación bajo Prueba:** Microservicio web Java estructurado con Spring Boot, clonado desde el repositorio oficial <https://github.com/springframeworkguru/springbootwebapp>.
- **Despliegue Local:** Compilado e instanciado de forma explícita bajo el puerto local 8083 mediante el comando: `java -jar target/spring-boot-web-0.0.1-SNAPSHOT.jar --server.port=8083`.
- **Herramienta de Inyección de Carga:** Apache JMeter v5.6.3 en modo binario ejecutable independiente.

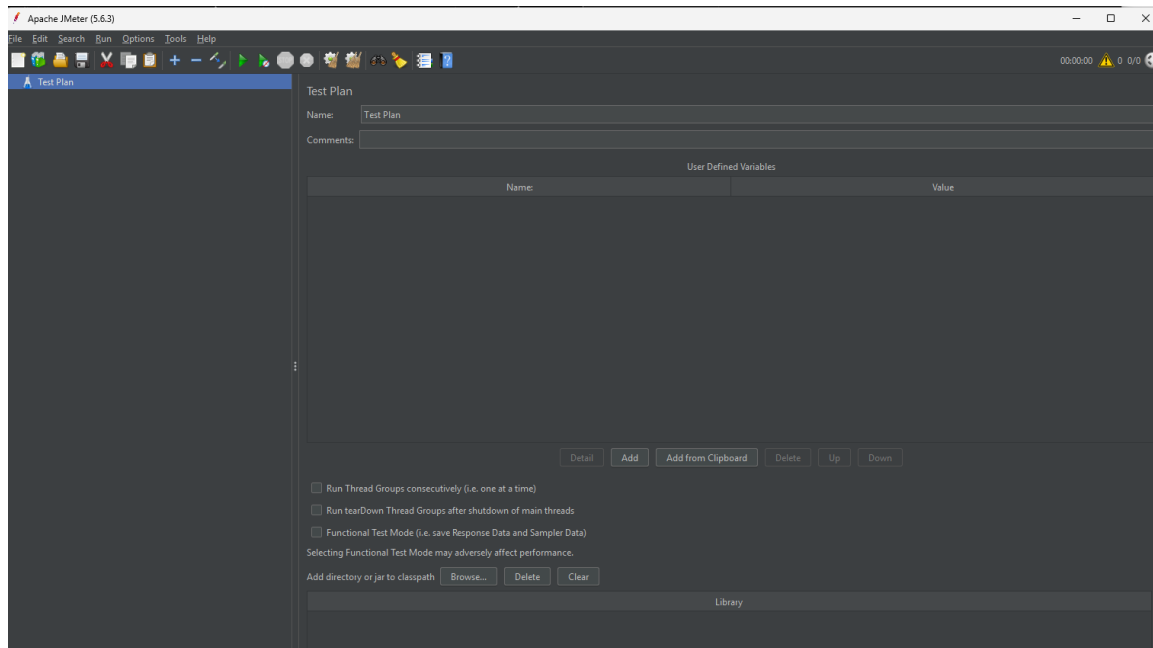


Figura 1: Inicialización y verificación de la suite Apache JMeter 5.6.3.

3 Diseño Metodológico de los Casos de Prueba

Se ha formulado un plan de carga compuesto por escenarios diferenciados con el fin de cubrir tanto consultas atómicas como inserciones de datos persistentes:

Tabla 1: Matriz de Casos de Prueba y Criterios de Aceptación de Rendimiento.

ID	Escenario Evaluado	Path HTTP	Método	Hilos	Límite Tolerable
TC1	Listado de Productos	/products	GET	100	≤ 1000 ms
TC2	Creación de Producto	/product	POST	300	≤ 3000 ms
TC3	Actualización de Registro	/product	POST/PUT	300	≤ 3000 ms

4 Implementación del Script Base y Validación Preliminar (TC1)

Se creó el archivo estructural `TestProductWeb.jmx`. Para mitigar anomalías por errores sintácticos en las peticiones HTTP, se ejecutó una fase previa de calibración con un solo hilo funcional. El sampler estructurado envió una solicitud GET hacia la ruta base de catálogo. La respuesta fue examinada en el módulo de visualización, confirmando un código de retorno 200 OK y payload correcto.

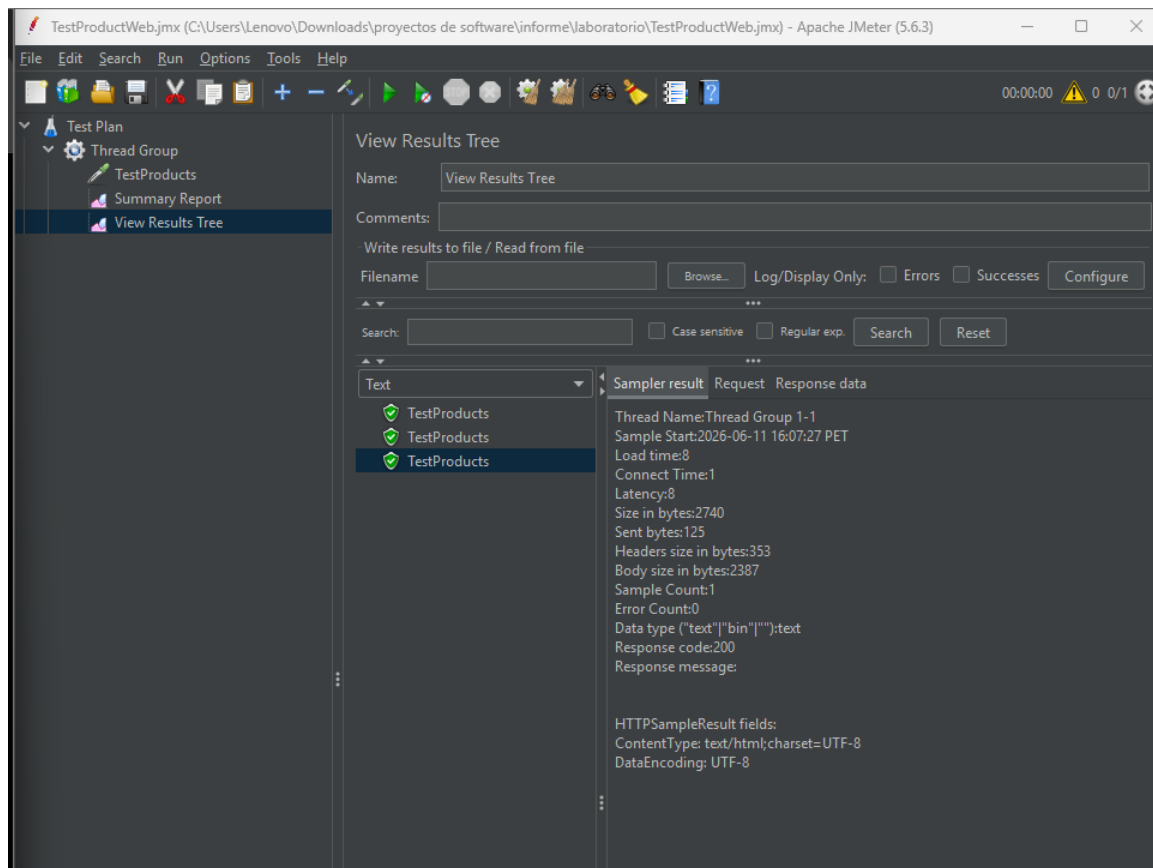


Figura 2: Validación de consistencia y código de estado exitoso 200 OK en View Results Tree.

5 Configuración de Escenarios Complejos y Cláusulas de Aserción (TC2)

Para emular interacciones realistas con formularios transaccionales, el escenario **TC2** incorporó un payload tipificado en el cuerpo de la petición POST. Se interceptaron y mapearon los campos obligatorios del backend: `productId`, `description`, `price` e `imageUrl`.

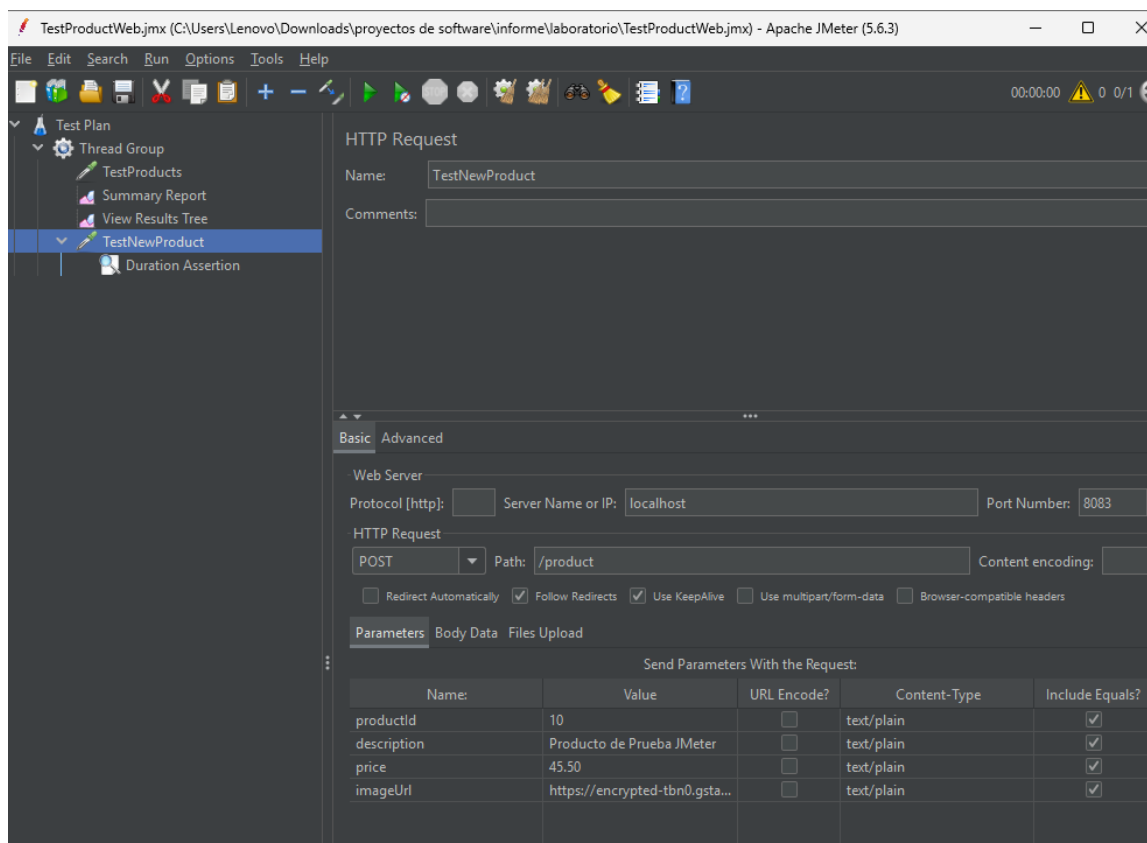


Figura 3: Definición formal de los parámetros clave en la solicitud transaccional POST.

Con el fin de automatizar la detección de la degradación del servicio, se inyectó una regla de validación de negocio basada en el tiempo (**Duration Assertion**). Esta cláusula intercepta las respuestas del hilo e invalida programáticamente cualquier solicitud cuyo procesamiento consuma más de 3000 ms.

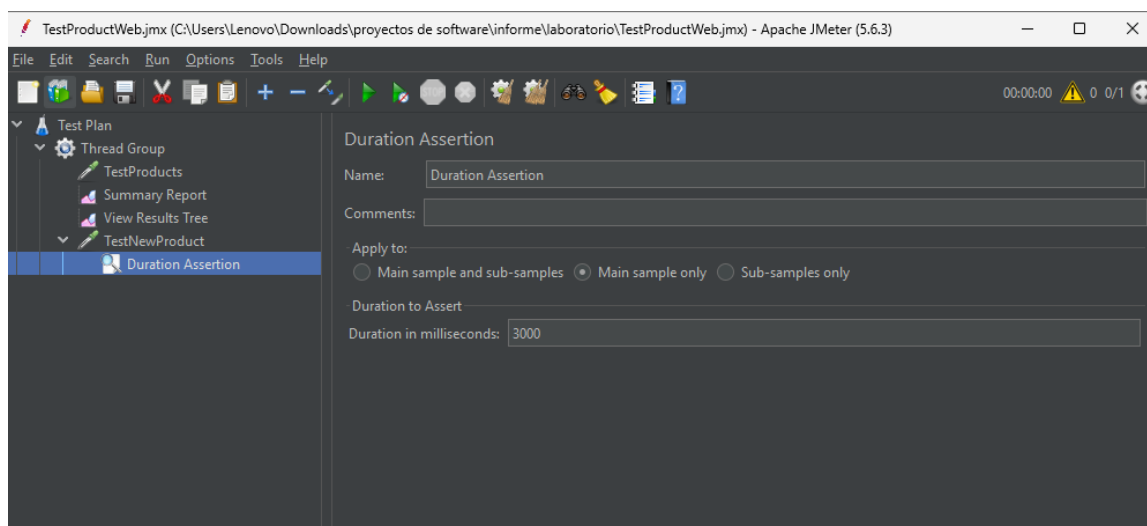


Figura 4: Inyección del componente Duration Assertion parametrizado en 3000 ms.

6 Ejecución Masiva, Monitoreo y Consolidación de Datos

Una vez certificado el flujo atómico, se escaló el Grupo de Hilos a cargas masivas concurrentes (de 100 a 300 hilos simulados con un factor de rampa proporcional). Se realizó un seguimiento

dinámico de los subprocesos de ejecución paralelos desde la consola de estado.

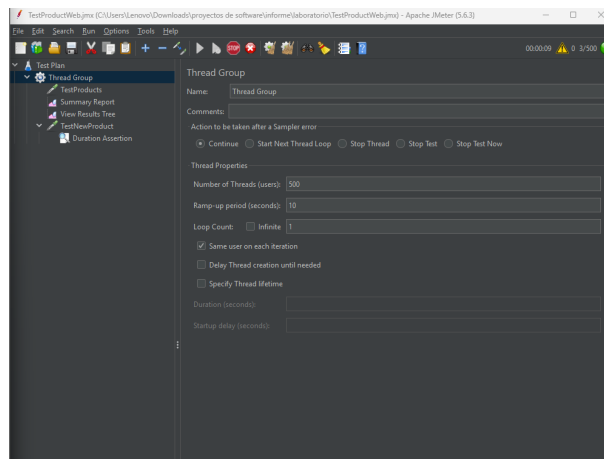


Figura 5: Contador en tiempo real de hilos concurrentes activos atacando la aplicación.

Los resultados recolectados se canalizaron hacia un acumulador estadístico estructurado (*Summary Report*), computando métricas operacionales de variabilidad como la mediana, rendimiento por segundo (Throughput) y tasa de descarte por errores.

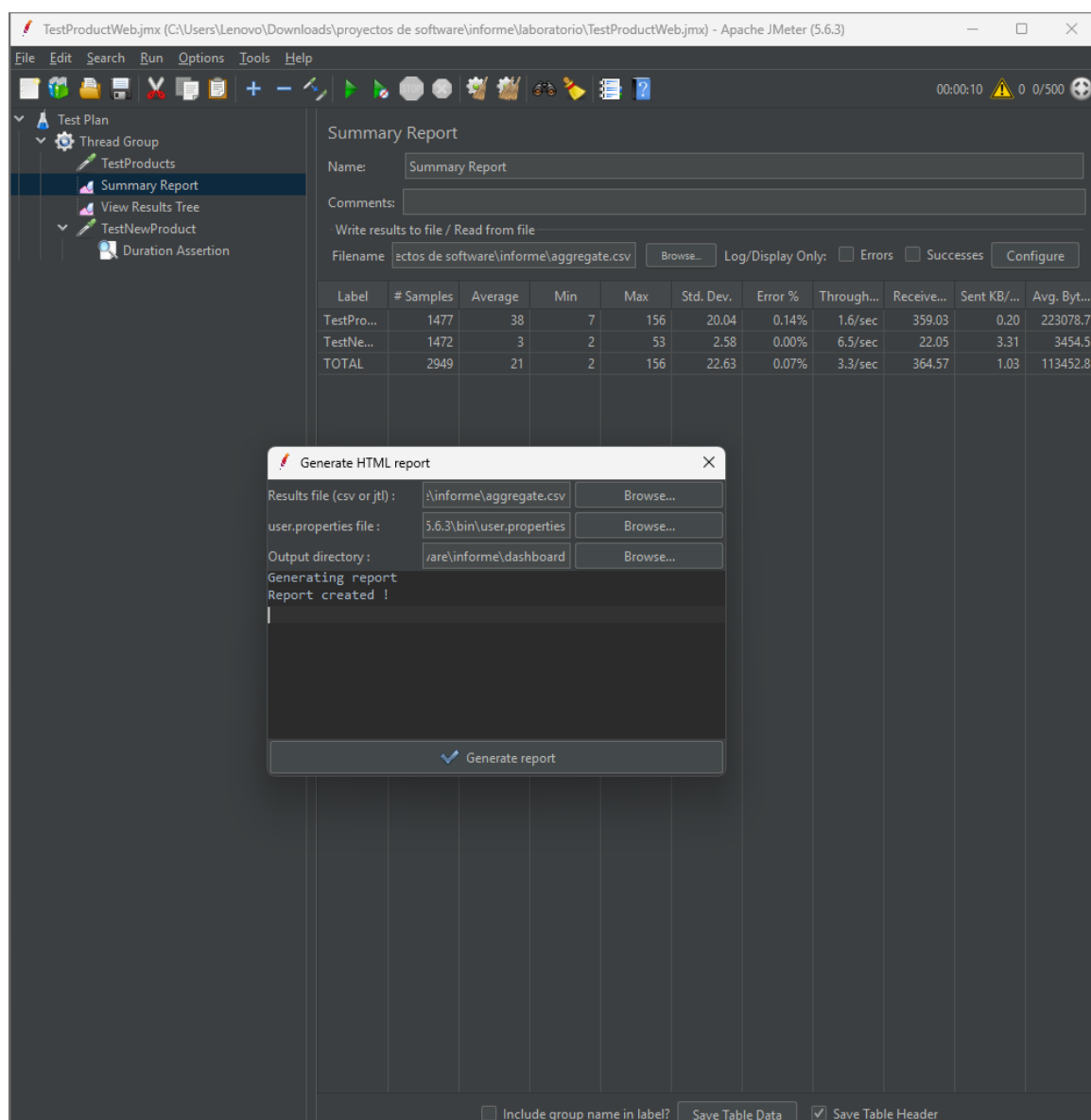


Figura 6: Compilación estadística global obtenida a través del Summary Report.

7 Análisis Avanzado mediante Dashboard HTML (Apdex)

Como entregable principal para la toma de decisiones, las muestras binarias se exportaron a un formato plano delimitado y se compilaron por medio del motor analítico de JMeter para estructurar el **Dashboard HTML Avanzado**. Este entorno web computa el índice de rendimiento de la aplicación (Apdex).

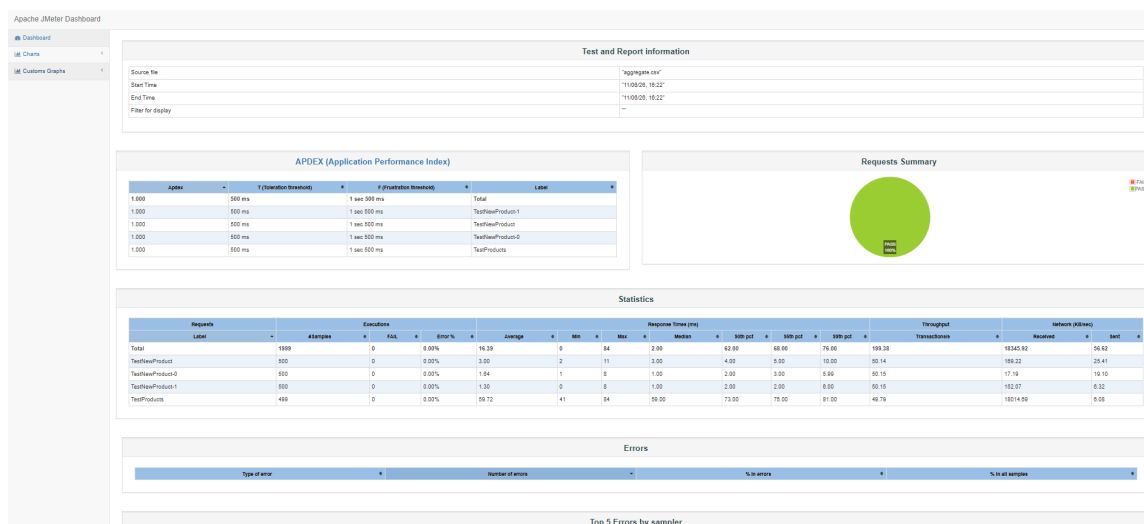


Figura 7: Vista ejecutiva del Dashboard Interactivo HTML y distribución porcentual del éxito.

Los datos indican un rendimiento óptimo. La aplicación Spring Boot demostró resiliencia arquitectónica frente a la concurrencia inyectada, asimilando las ráfagas transaccionales dentro de los límites de tolerancia prefijados sin experimentar saturación de memoria ni fallos de timeout.

8 Conclusiones y Recomendaciones

- **Eficiencia de la Suite:** La automatización mediante scripts `.jmx` mitiga la variabilidad de las pruebas manuales y maximiza la cobertura sobre endpoints de lógica transaccional.
- **Robustez del Sistema:** La arquitectura Spring Boot bajo el puerto 8083 gestionó la carga transaccional de forma balanceada, cumpliendo los criterios de aceptación sin elevación crítica del porcentaje de error.
- **Recomendación de Infraestructura:** Se aconseja incorporar monitores de consumo de hardware (CPU/RAM) en futuras iteraciones para correlacionar los picos de latencia con cuellos de botella a nivel de sistema operativo.